

Introduction to Django Web Framework

Core concepts, architecture design patterns, project vs. app structure, and setup guidelines.

1. What is Django?

Django is a high-level, open-source **Python web framework** designed to enable the rapid development of secure, scalable, and maintainable websites. Built by experienced developers, it takes care of much of the hassle of web development, allowing you to focus on writing your app without needing to reinvent the wheel.

"Batteries Included" Philosophy

Django ships out-of-the-box with common tools: User Authentication, Admin Interface, ORM (Database integration), URL routing, and Security features (CSRF, XSS prevention).

DRY Principle (Don't Repeat Yourself)

Encourages reusability and efficiency. Every distinct piece of knowledge or logic should have a single, unambiguous representation within a system.

2. Real-World Applications

Django powers major modern web platforms due to its scalability and security standards:

- **Content Management & Publishing:** High-traffic media platforms (e.g., The Washington Post).
- **Social Media & Sharing:** Platforms requiring robust API backends (e.g., Instagram, Pinterest).
- **Data & Analytics:** Scientific and fintech applications leveraging Python's rich AI/ML ecosystems.

3. Development Prerequisites & Setup Setup

Before launching a Django project, developers establish an isolated execution environment:

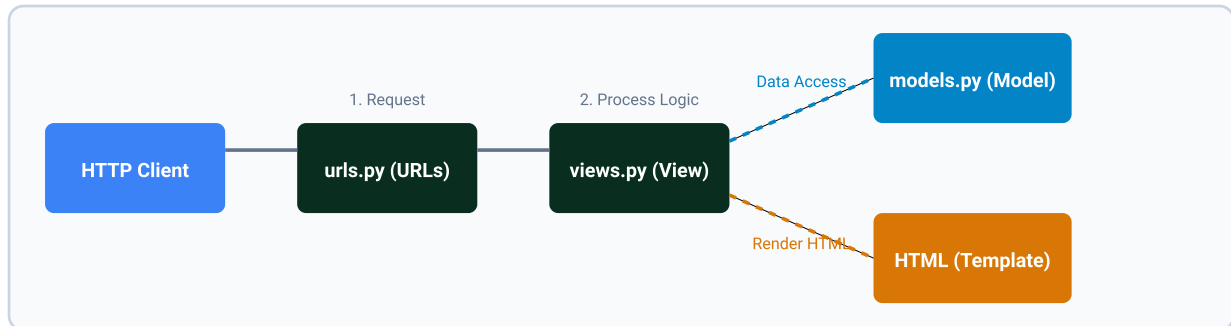
```
# 1. Create a virtual environment python -m venv myenv # 2. Activate environment  
(Windows) myenv\Scripts\activate # Activate environment (Mac/Linux) source myenv/bin/activate  
# 3. Install Django within virtual environment pip install django
```

Why Virtual Environments?

Virtual environments isolate dependencies per project, preventing version conflicts between different Python software packages across your operating system.

4. Django's Architecture: MVT Pattern

Django follows the **Model-View-Template (MVT)** design pattern, a software design variation of the classic Model-View-Controller (MVC) pattern.



Component	Role & Function	Key File Reference
Model	Defines the data structure, database tables, and logic. Interacts directly with database using Python ORM.	models.py
View	Contains business logic. Receives HTTP requests, fetches data via Models, and renders HTTP responses.	views.py
Template	Handles presentation. Dynamic HTML files mixed with Django Template Language (DTL).	*.html files

5. Tracing the Request-Response Lifecycle

- HTTP Request:** User enters a URL in their web browser.
- URL Dispatcher (urls.py):** Django inspects the URL pattern and matches it to the appropriate view function/class.
- View Handler (views.py):** The view processes business rules. It queries data via **Model** if needed.
- Database interaction (models.py):** Model translates Python code to SQL and fetches standard objects.
- Template Rendering:** View takes the dataset, injects it into a **Template**, and compiles an HTML document.
- HTTP Response:** The View sends the rendered response back to the browser.

6. Projects vs. Applications

A fundamental Django architectural concept is distinguishing between a Project and an App.

Django Project

The entire web application and global configurations. Contains global settings, database choices, and root URL configurations.

Analogy: An entire website (e.g., e-commerce platform).

Django App

A self-contained Python package that performs a specific feature or function. Apps can be reused across different projects.

Analogy: A module (e.g., Payment system, Blog, Polls).

7. Core CLI Commands Reference

Command	Description
<code>django-admin startproject <name></code>	Creates the outer directory structure and boilerplate configuration files for a new project.
<code>python manage.py startapp <app_name></code>	Generates the default module directory containing <code>models.py</code> , <code>views.py</code> , and <code>admin.py</code> .
<code>python manage.py runserver</code>	Starts the local development server (defaults to <code>http://127.0.0.1:8000/</code>).
<code>python manage.py makemigrations</code>	Prepares migration files based on changes detected in <code>models.py</code> .
<code>python manage.py migrate</code>	Applies pending database migrations and updates schema tables.

8. Anatomy of a Django Directory Structure

```
myproject/ | ├── manage.py # CLI tool for project tasks └── myproject/ # Global Project
Settings Package ├── __init__.py # Marks directory as a Python package └── settings.py #
Central config (DBs, installed apps) ├── urls.py # Global URL declarations └── asgi.py #
Entry point for ASGI web servers └── wsgi.py # Entry point for WSGI web servers | └── myapp/
# Individual Application Module ├── __init__.py ├── admin.py # Register models with Django
admin ├── apps.py # App-specific configuration ├── migrations/ # Database migration history
└── models.py # Data models schema └── tests.py # Automated unit tests └── views.py #
Business logic & endpoints
```